Scalable Simulation-Based Inference With Optimization Monte Carlo

Anonymous Author
Anonymous Institution

Abstract

Computer simulators are widely used to understand and predict the behavior of complex stochastic systems. Inferring the parameters of such simulators from data is difficult due to the intractability of their likelihood function. A range of simulation-based methods have been introduced to extend Bayesian inference to simulators, but they tend to struggle when the parameter space is high-dimensional, when the data consists of multiple iid observations, and when only a small part of the simulator output relates to the parameters of interest. Here, we propose a gradient-based method, building on the framework of robust optimization Monte Carlo, to navigate the high-dimensional parameter space and find parameter configurations that can plausibly explain multiple iid observations. The use of gradients in our approach further enables us to perform sensitivity tests and filter out dimensions that are not informative of the parameters. We offer an efficient `JAX`-based implementation that parallelizes key parts of the method. Extensive experiments show that our approach efficiently generates accurate posterior samples, even in challenging scenarios with high-dimensional parameters, multiple observations, and uninformative outputs, where most existing methods fall short.

1 Introduction

Parametric stochastic simulators are widely used to model complex processes in biology, physics, the health and climate sciences, and many more disciplines. As these simulators become more accurate, they become more predictive. However, this improvement often comes at the cost of greater model complexity—introducing more variables, a large number of free parameters, and a higher-dimensional output space. This increased complexity poses significant challenges for parameter inference.

A range of methods have been developed to infer parameters of simulators. They are known as Likelihood-Free (LFI) or Simulation-Based (SBI) Inference methods (Lintusaari et al., 2017; Sisson et al., 2018; Cranmer et al., 2020) and involve sampling-based approaches as in Approximate Bayesian Computation (ABC, Marin et al., 2012), the construction of summary statistics (Chen et al., 2021, 2023), their parametric modeling (Wood, 2010; Price et al., 2018), ratio estimation (Thomas et al., 2022; Durkan et al., 2020), as well as neural surrogate models for the likelihood (NLE, Papamakarios et al., 2019) or the posterior (NPE, Greenberg et al., 2019; Papamakarios and Murray, 2016).

Parameter inference becomes more challenging when the parameter and data space are high-dimensional. SBI methods tend to be particularly affected because they rely on populating the parameter and data space with samples (Lintusaari et al., 2017; Sisson et al., 2018; Cranmer et al., 2020), so that the curse of dimensionality becomes notable. For complex simulators, where many parameters and variables interact in intricate ways, some variables may only depend weakly, or not at all, on a subset of parameters (Saltelli et al., 2008; Monsalve-Bravo et al., 2022). This can lead to situations where some observable variables are not informative of the parameters of interest and become "distractors" during parameter inference (Lueckmann et al., 2021). Multiple iid observations can exacerbate issues in SBI since the methods need to find parameter configurations that not only explain one observation, which is already difficult, but multiple ones at the same time. Figure **??** illustrates the difficulty of existing SBI methods with high-dimensional parameter spaces, the presence of distractors, multiple iid observations, or all of them combined.

We here propose a new method that uses gradient information to deal with the above issues. We first convert the stochastic simulator to a set of deterministic

Preliminary work. Under review by AISTATS 2025. Do not distribute.

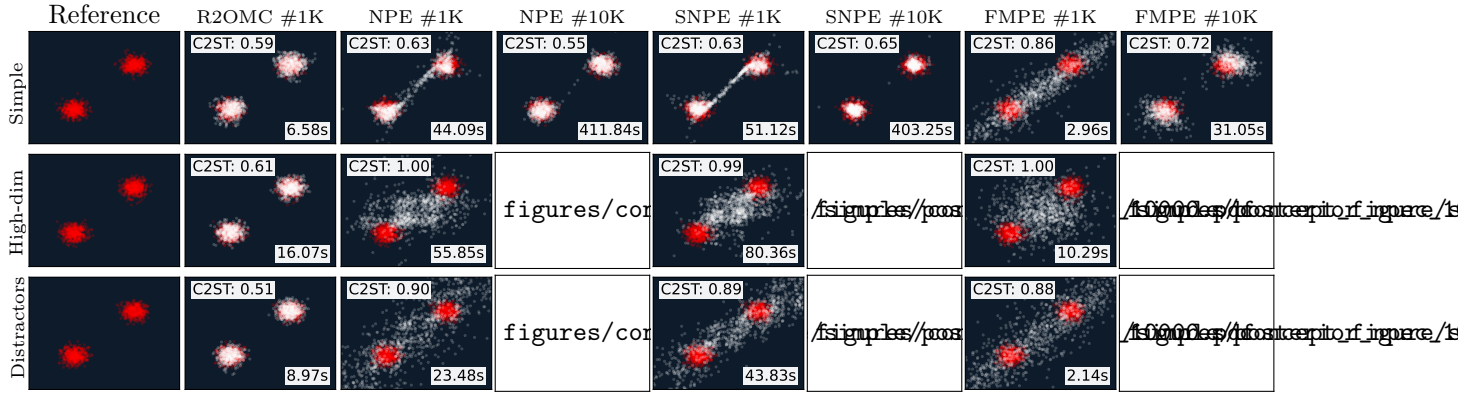


Figure 1: Your figure caption here.

simulators following the framework of Robust Optimization Monte Carlo (ROMC, Meeds and Welling, 2015; Ikonomov and Gutmann, 2020) and then use gradient-descent to efficiently obtain posterior samples. Very much like in standard optimization, the gradients are key to navigate the high-dimensional parameter space (Figure **??**, columns 3-4). In addition, they allow us to perform sensitivity tests and filter out distractor dimensions, thereby improving the quality of the inference.

The ROMC framework provides the theory to convert posterior inference to deterministic optimization, which enables scalability to high-dimensional parameters, but is silent on the issues of multiple iid observations and the presence of distractors. In particular, it requires us to define a suitable distance function between simulated and observed data, and this becomes difficult with multiple observations or distractors. Naive approaches can result in poor approximations of the posterior, as illustrated in Figure **??** (column 3, rows 3-5). Moreover, its latest implementation (Gkolemis et al., 2023) does not exploit gradients and hence does not scale well to high-dimensional parameter dimensions.

We here overcome these challenges, and introduce R2OMC, an SBI method that belongs to the ROMC family of methods, which

- enables high-dimensional gradient-based sampling from the posterior parameter distribution,
- handles multiple iid observations,
- automatically identifies and filters out uninformative dimensions (distractors),
- runs efficiently even for high-dimensional problems due to a `JAX` implementation that uses automatic differentiation, vectorization, and just-in-time compilation.

R2OMC is systematically compared to state-of-the-art SBI methods. We find that it delivers accurate posterior samples at a fraction of the cost, even for problems where most SBI methods struggle.

2 Background and Motivation

We here briefly introduce Robust Optimization Monte Carlo (ROMC) and discuss the difficulties due to multiple iid data points and distractors.

2.1 ROMC framework

ROMC has been introduced by Ikonomov and Gutmann (2020), extending and addressing robustness issues of the Optimization Monte Carlo method by Meeds and Welling (2015). It belongs to the larger family of ABC methods that approximate the likelihood function as the probability of simulating an output ϵ-close to the observation,

$$L_\epsilon(\boldsymbol{\theta}) = \Pr(d(\mathbf{y}, \mathbf{y}^o) \leq \epsilon \mid \boldsymbol{\theta}). \tag{1}$$

Here, $d(\cdot, \cdot)$ is a distance, $\mathbf{y}$ the simulator output, and $\mathbf{y}^o$ the observed data. In the following, we denote by $g(\boldsymbol{\theta}, \mathbf{u})$ the simulator that maps noise (nuisance) variables $\mathbf{u} \sim p(\mathbf{u})$[1] and parameters $\boldsymbol{\theta}$ to multivariate data $\mathbf{y} = g(\boldsymbol{\theta}, \mathbf{u})$. Introducing the acceptance region $C_\epsilon = \{(\boldsymbol{\theta}, \mathbf{u}) : d(g(\boldsymbol{\theta}, \mathbf{u}), \mathbf{y}^o) \leq \epsilon\}$ and using the law of the unconscious statistician, (1) can be written as

$$L_\epsilon(\boldsymbol{\theta}) = \mathbb{E}_{p(\mathbf{u})}\left[\mathbb{1}_{C_\epsilon}(\boldsymbol{\theta}, \mathbf{u})\right], \tag{2}$$

where $\mathbb{1}_{C_\epsilon}$ denotes the indicator function that is one if $(\boldsymbol{\theta}, \mathbf{u}) \in C_\epsilon$ and zero otherwise. When approximating the expectation as a sample average, we obtain

$$L_\epsilon(\boldsymbol{\theta}) \approx \frac{1}{S} \sum_{i=1}^{S} \mathbb{1}_{C_\epsilon^i}(\boldsymbol{\theta}), \tag{3}$$

[1] In a computer program, sampling $\mathbf{u}$ corresponds to sampling a random seed.

which is the proportion of acceptance regions $C_\epsilon^i = \{\boldsymbol{\theta} : d(g(\boldsymbol{\theta}, \mathbf{u}_i), \mathbf{y}^o) \leq \epsilon\}$ that contain $\boldsymbol{\theta}$.

ROMC takes (3) as starting point and focuses on characterizing the acceptance regions C_ϵ^i, because once the C_ϵ^i are known, we can not only evaluate the approximate likelihood function, but also obtain posterior samples efficiently. ROMC achieves this by introducing (uniform) proposal distributions q_i with support on C_ϵ^i only. Samples from $\boldsymbol{\theta}_{ij} \sim q_i$ for $j \in 1, \ldots, M$, are then equal to posterior samples if weighted with the (unnormalized) weights w_{ij}, $i = 1, \ldots, S, j = 1, \ldots, M$,

$$w_{ij} = \frac{p(\boldsymbol{\theta}_{ij})}{q_i(\boldsymbol{\theta}_{ij})} \mathbb{1}_{C_\epsilon^i}(\boldsymbol{\theta}_{ij}). \tag{4}$$

For the characterization of the acceptance regions C_ϵ^i, ROMC first determines the minimizers $\boldsymbol{\theta}_i^*$,

$$\boldsymbol{\theta}_i^* = \text{argmin}_{\boldsymbol{\theta}} d_i(\boldsymbol{\theta}), \quad d_i(\boldsymbol{\theta}) = d(g(\boldsymbol{\theta}, \mathbf{u}_i), \mathbf{y}^o), \tag{5}$$

where $d_i(\boldsymbol{\theta})$ are deterministic distance functions because the sources of randomness (seed) are set to $\mathbf{u}_i$ and kept fixed. Next, it starts from the optimization end points $\boldsymbol{\theta}_i^*$ to map out C_ϵ^i using for ϵ a value derived from the quantiles of the minimial distance functions, $d_i^* = d_i(\boldsymbol{\theta}_i^*), i = 1, \ldots, S$ (see Ikonomov and Gutmann, 2020, for details). While several options are possible (Ikonomov and Gutmann, 2020), we here use hyperboxes centered at $\boldsymbol{\theta}_i^*$, which can be constructed with a line search algorithm for each parameter dimension. For details, please see Appendix A.

The power of the ROMC framework stems from the possibility to use optimization, in particular gradient-based methods, to determine the optimization end points $\boldsymbol{\theta}_i^*$, and hence the acceptance regions C_ϵ^i and the proposal distributions q_i. We will exploit this in our method to scale SBI to higher parameter dimensions. Before we can tap into the ROMC framework, however, we will need to overcome challenges related to the definition of the distance function d in case of multiple iid observations and the presence of distractors.

2.2 Challenges due to iid data and distractors

The ROMC framework is silent on the choice of the distance function d. It assumes that a suitable one has been defined. However, poor choices can be detrimental to the posterior approximation. In particular, if the minimal distances d_i^* remain too large for many seeds i even after successful optimization, ϵ will be chosen too large and the posterior approximation will be too loose. We here highlight two important cases where defining a suitable distance function $d_i(\boldsymbol{\theta})$ is difficult, both in general and for ROMC in particular.

Multiple iid observations. Consider a simulator $\mathbf{y}|\boldsymbol{\theta} \sim 0.5\,(\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I}))$, where $\boldsymbol{\theta} \in \mathbb{R}^{10}$ are the parameters of interest, a uniform prior on $[-15, 15]^{10}$ for them, and $N = 4$ iid observations. Let the first two observed data points $\mathbf{y}^o_{\{1,2\}} \approx (5, \ldots, 5)$ and the last two $\mathbf{y}^o_{\{3,4\}} \approx (-5, \ldots, -5)$. The posterior for $\boldsymbol{\theta}$ is approximately a truncated mixture of Gaussians whose components are centered at $(5, \ldots, 5)$ and $(-5, \ldots, -5)$ with a covariance matrix equal to $\mathbf{I}/4$ (see Appendix B).

A straightforward approach for defining the distance function is to run the simulator with different seeds $\mathbf{u}_{i,n}$ for each observation n, and to average the per data-point distances so that $d_i(\boldsymbol{\theta}) = 1/N \sum_n \tilde{d}(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}), \mathbf{y}^o_n)$, where $\tilde{d}$ is a distance function for a single data point. However, this approach is sensitive to the ordering of the observations. The optimal distance d_i^* approaches zero only when $\mathbf{u}_{i,1}, \mathbf{u}_{i,2}$ make the simulator generate data from the same mixture component, say $\mathcal{N}(\boldsymbol{\theta}, \mathbf{I})$, and $\mathbf{u}_{i,3}, \mathbf{u}_{i,4}$ from the opposite one, say $\mathcal{N}(-\boldsymbol{\theta}, \mathbf{I})$). In all other scenarios, d_i^* remains large, resulting in a large ϵ that leads to a posterior that is broader than the true posterior. This issue increases with the number of observations due to the factorial growth in possible permutations. Similar issues occur for other simple approaches such as concatenating all the iid data points into a single vector. For squared Euclidean distances, the two approaches coincide. Figure **??** illustrates the drop in performance of ROMC when averaging per-data point distances.

Uninformative dimensions (distractors). We defined distractors to be output dimensions that do not relate to the parameters of interest. A concrete example is a graphics renderer where the parameter of interest controls a localized part of the scene, e.g. the appearance of vase, so that most pixel values of the scene are unrelated to the parameter. To emulate distractors, we consider a simulator $\mathbf{y} = (\mathbf{y}^{(1)}, \mathbf{y}^{(2)})$, where the first set of dimensions $\mathbf{y}^{(1)} \sim 0.5\,(\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I}))$ and the second set of dimensions $\mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I})$, where $\boldsymbol{\mu} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$. Note that only $\mathbf{y}^{(1)}$ depends on $\boldsymbol{\theta}$; $\mathbf{y}^{(2)}$ represents distractors. Given one observation $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})$, with $\mathbf{y}^{o,(1)} \approx (5, \ldots, 5)$, and for $\boldsymbol{\theta}$ a uniform prior on $[-15, 15]^D$ as before, the posterior is solely influenced by $\mathbf{y}^{o,(1)}$. It is approximately equal to a truncated mixture of two Gaussians with components centered at $(5, \ldots, 5)$ and $(-5, \ldots, -5)$, each having an identity covariance matrix, see Appendix B. Since $\mathbf{y}^{(2)}$ does not depend on $\boldsymbol{\theta}$, the minimal distances d_i^* will generally be large for seeds that generate an output $\mathbf{y}^{(2)}$ significantly different from $\mathbf{y}^{o,(2)}$. This results in a large ϵ, which again leads to a posterior that is much broader than the true posterior. Figure **??** visualizes how ROMC is affected by distances that are sensitive to distractors.

3 Proposed method: R2OMC

We here present our new method for Bayesian parameter inference in simulator models. The method uses gradients to obtain posterior samples even in high-dimensional parameter spaces. The method belongs to the ROMC framework so that we start with discussing how we deal with iid observations and distractors before discussing its implementation.

3.1 Multiple iid observations

The likelihood for multiple iid observations $\{\mathbf{y}_n^o\}_{n=1}^N$ is the product of the likelihoods for each data point. Approximating the likelihood for each data point as in (3) and multiplying-in the prior $p(\boldsymbol{\theta})$, we can obtain a formal expression for the approximate posterior

$$p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\}) \propto p(\boldsymbol{\theta}) \prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}), \tag{6}$$

where $C_\epsilon^{i,n}$ is the acceptance region for data point n and source of randomness (seed) $\mathbf{u}_{i,n}$,

$$C_\epsilon^{i,n} = \{\boldsymbol{\theta} : d(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}), \mathbf{y}_n^o) \leq \epsilon\}. \tag{7}$$

We have S sources of randomness for each data point $\mathbf{y}_n^o$, and so in total $S * N$ acceptance regions $C_\epsilon^{i,n}$.

We now address the question how to efficiently sample from $p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\})$. To achieve this, we consider the task of computing the posterior expectation $\bar{h} = \mathbb{E}_{p_\epsilon}[h(\boldsymbol{\theta})]$ under $p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\})$. Plugging in the expression in (6) gives, after normalization,

$$\bar{h} = \frac{\int h(\boldsymbol{\theta}) p(\boldsymbol{\theta}) \prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) d\boldsymbol{\theta}}{\int p(\boldsymbol{\theta}) \prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) d\boldsymbol{\theta}}. \tag{8}$$

The integrals in the equation are typically intractable. But they correspond to expectations with respect to the prior $p(\boldsymbol{\theta})$ and hence could be approximated with a sample average. However, this is very inefficient. Few (or no) prior samples will generate outputs ϵ-close to all observations, so most (or all) will get zero weight, leading to a very low effective sample size. Following the general ROMC framework by Ikonomov and Gutmann (2020), we thus employ importance sampling with an adaptive proposal distribution obtained via optimization.

We first treat each data point $\mathbf{y}_n^o$ separately and construct proposal distributions $q_i^n(\boldsymbol{\theta})$, $i = 1, \ldots, S$, as in Section 2.1: We use optimization to find the minimizers $\boldsymbol{\theta}_{i,n}^*$ of the deterministic functions $d_i^n(\boldsymbol{\theta}) = d(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}), \mathbf{y}_n^o)$,

$$\boldsymbol{\theta}_{i,n}^* = \text{argmin}_{\boldsymbol{\theta}} d_i^n(\boldsymbol{\theta}), \tag{9}$$

and then build a hyperbox around each acceptance region $C_\epsilon^{i,n}$ and define $q_i^n(\boldsymbol{\theta})$ to be the uniform distribution over it. The optimization problem in (9) is deterministic and we solve it with gradient-based methods to enable scalability to large parameter spaces. Note that treating each data point $\mathbf{y}_n^o$ separately allows us to avoid the issues explained in Section 2.2.

We propose to combine the per-data point proposal distributions $q_i^n(\boldsymbol{\theta})$ in form of a mixture distribution,

$$q(\boldsymbol{\theta}) = \frac{1}{N}\frac{1}{S} \sum_{n=1}^N \sum_{i=1}^S q_i^n(\boldsymbol{\theta}). \tag{10}$$

The posterior expectation in (8) can then be approximated in form of a weighted average

$$\bar{h} = \frac{\sum_p w_p h(\boldsymbol{\theta}_p)}{\sum_p w_p}, \quad \boldsymbol{\theta}_p \sim q(\boldsymbol{\theta}) \tag{11}$$

where the weights w_p, $p = 1, \ldots, P$, are given by

$$w_p = \frac{p(\boldsymbol{\theta}_p)}{q(\boldsymbol{\theta}_p)} \prod_{n=1}^N \sum_{i=1}^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}_p). \tag{12}$$

This means that the $\boldsymbol{\theta}_p \sim q(\boldsymbol{\theta})$, weighted by w_p, represent samples from the posterior $p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\})$ in (6).

The rationale for choosing $q(\boldsymbol{\theta})$ in (10) as proposal distribution is as follows: it is a mixture of uniform distributions, so sampling and evaluating is easy. It seamlessly combines the contribution from multiple data points, allowing future data points to be included without re-computation. If each q_i^n encloses the corresponding acceptance region $C_\epsilon^{i,n}$, then $q(\boldsymbol{\theta}) > 0$ in areas of the parameter space where $\prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) > 0$ and no posterior mass is discarded in the weighting in (12). Finally, regions of high likelihood are characterized by regions where multiple $C_\epsilon^{i,n}$ overlap. As the proposal distribution $q(\boldsymbol{\theta})$ consists of a mixture of the $q_i^n(\boldsymbol{\theta})$ with equal weights, areas where multiple $C_\epsilon^{i,n}$ overlap will be sampled more often, so that areas of high likelihood will be represented with more samples.

Figure 2, steps (ii-iii), illustrates the proposed procedure to obtain posterior samples. We have two observed data points and the $\boldsymbol{\theta}_p$ come from regions that match at least one observation; the blue-dotted area for $\mathbf{y}_1^o$ and the green-dotted region for $\mathbf{y}_2^o$. However, only the ones that match both will get a positive weight (red dots) and become posterior samples.

3.2 Uninformative dimensions (distractors)

Distractors can negatively affect the quality of the posterior approximation. To counteract their influence, we measure the sensitivity of the output dimensions

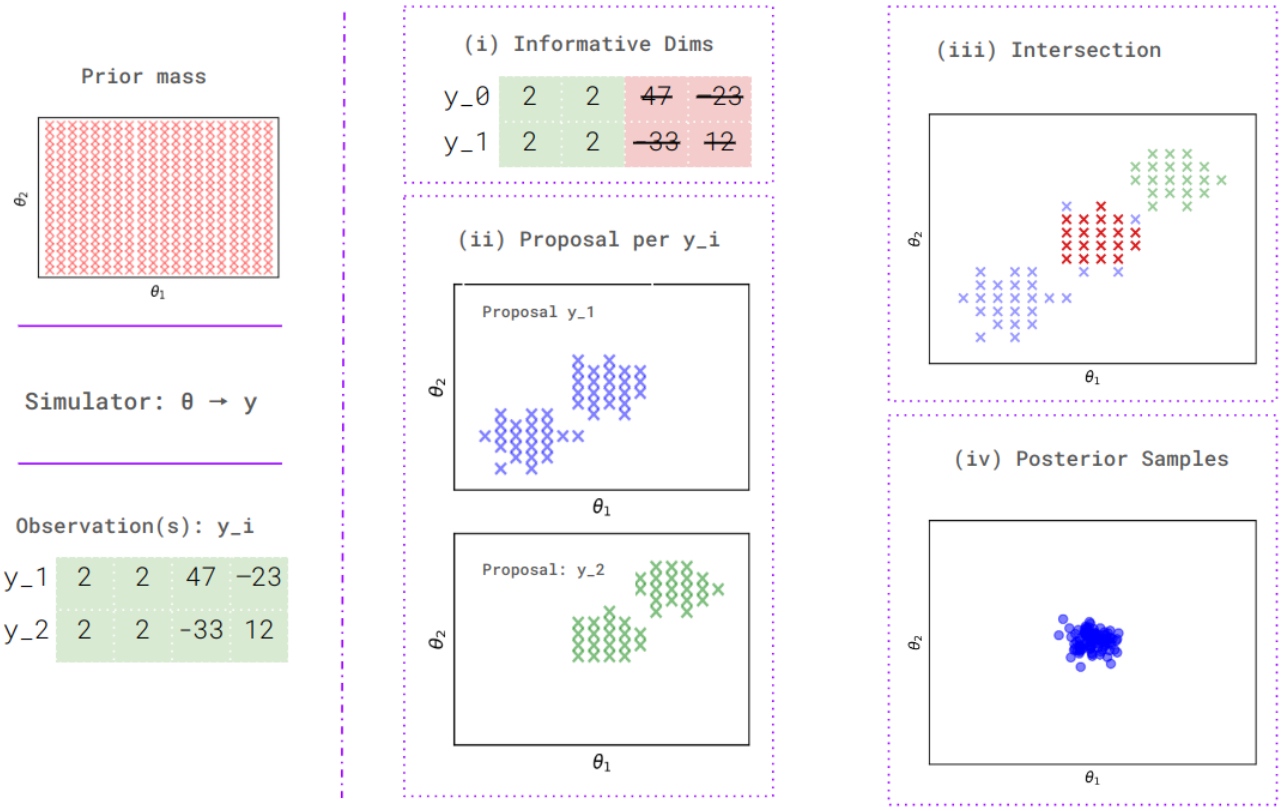


Figure 2: Overview R2OMC: Distractors are automatically filtered out and proposal distributions q_i^n are created for each observation (here two): the blue-dotted region indicates where the proposal distributions for the first observation are nonzero, and the green-dotted region indicates the same for the second observation. Only samples that fall within the intersection of both regions are given a positive weight and become posterior samples.

with respect to the parameters $\boldsymbol{\theta}$ by computing the average norm of their derivative with respect to $\boldsymbol{\theta}$, and checking that the value is above a minimal value (threshold) τ,

$$I_i = \mathbb{E}_{p(\boldsymbol{\theta})p(\mathbf{u})}\left[||\nabla_{\boldsymbol{\theta}} g_i(\boldsymbol{\theta}, \mathbf{u})||\right] > \tau. \tag{13}$$

The expectation equals the expectation with respect to the joint distribution of the prior and the i-th dimension of the simulator output, $g_i(\boldsymbol{\theta}, \mathbf{u})$. In our experiments, we approximate the expectation using 50 samples from $p(\boldsymbol{\theta})$ and $p(\mathbf{u})$ each. Collecting all binary indicator variables I_i into the vector $\mathbf{m}_\tau$, we can filter out uniformative dimensions by computing the distance between observed and simulated data for informative dimensions only. To compute $\boldsymbol{\theta}_{i,n}^*$ in (9), we minimize

$$d_i^n(\boldsymbol{\theta}) = d(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}) \odot \mathbf{m}_\tau, \mathbf{y}_n^o \odot \mathbf{m}_\tau). \tag{14}$$

The procedure is illustrated in Figure 2, step (i).

The threshold τ controls the sensitivity of the test. In our experiments, we set τ to machine precision; more sophisticated approaches consist in monitoring the distribution of the expected norm of the gradient, or by assessing false vs true positives under control conditions. This is possible since the mask $\mathbf{m}_\tau$ can be computed prior to receiving observed data.

3.3 `JAX` implementation

Algorithms 1 summarizes the key steps of R2OMC. It has three steps: dealing with distractors (line 2); constructing per-data point proposal distributions $\{q_i^n\}_{i=1}^S$ (lines 4-5); combining them to form the overall proposal distribution $q(\boldsymbol{\theta})$ and generating the weighted samples (lines 7-8).

We offer an efficient `JAX` implementation, benefiting from automatic differentiation (`grad`), vectorization (`vmap`), and just-in-time compilation (`jit`). In many cases, R2OMC executes in seconds where most SBI methods require minutes (see Section 4).

R2OMC requires approximately $N*S$ evaluations of the simulator: For the distractors, `grad` computes $\partial g_i / \partial \theta_j$ for all i and j in one call, and `vmap` evaluates multiple $\boldsymbol{\theta}$ at once. Combining them, the cost is S. Obtaining the per-data point proposal distributions $\{q_i^n\}_{i=1}^S$ consists of optimization (line 4) and hyperbox building (line 5)

Algorithm 1 `R2OMC`. Requires: $p(\boldsymbol{\theta}), g(\boldsymbol{\theta}, \mathbf{u}), \{\mathbf{y}_n^o\}_{n=1}^N$

1: **procedure** R2OMC(τ, S, P)
2: $\quad \mathbf{m}_\tau = (I_1, \ldots, I_{D_y})$ using (13) with τ
3: $\quad$ **for** $n \leftarrow 1$ to N **do**
4: $\quad\quad \boldsymbol{\theta}_{i,n}^* = \text{argmin}_{\boldsymbol{\theta}} d_i^n(\boldsymbol{\theta})$ using (14) for all i
5: $\quad\quad$ Compute hyperboxes and $\{q_i^n\}_{i=1}^S$
6: $\quad$ **end for**
7: $\quad \boldsymbol{\theta}_p \sim q(\boldsymbol{\theta})$, where $q(\boldsymbol{\theta}) = \frac{1}{N}\frac{1}{S}\sum_n \sum_i q_i^n(\boldsymbol{\theta})$
8: $\quad$ Compute $\{w_p\}_p^P$ using (12)
9: $\quad$ **return** $\{w_p, \boldsymbol{\theta}_p\}_{p=1}^P$
10: **end procedure**

for each data point. Each of S optimizations requires T gradient descent steps, where T is a user-defined hyperparameter. `jit` allows compiling T_1 consequtive steps to apply them at the cost of a single execution, so the cost reduces to $S\frac{T}{T_1}$. $\frac{T}{T_1}$ is normally small, and can even reach 1 if memory allows setting $T_1 = T$. Hyperbox building also requires of the order of S steps when the required line searches are parallelized over each parameter dimensions using `vmap` (see Appendix A). Thus the total cost of this step is of the order of $N * S$. For generating the weighted samples, (12) is evaluated on P samples (line 8). The simulator is only used for computing $\prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}_p)$ which requires evaluating $S * N$ different distance functions on P points each. Using `vmap` in each distance function, the total cost is $S * N$. Evaluating $q(\cdot)$ and $p(\cdot)$ does not require calling the simulator. Finally, if the simulator is expensive, the $S * N$ cost of checking whether the samples are in the acceptance regions can be avoided by using the hyperbox or another model for the acceptance region, see Ikonomov and Gutmann (2020).

The code is available at *anonymized link*.

4 Experiments

We report simulation results on a Mixture of Gaussians (MoG) benchmark in Section 4.1 and the SBI benchmark by Lueckmann et al. (2021) in Section 4.2. The MoG benchmark thoroughly evaluates methods under high-dimensionality, distractors and multiple observations. SBI considers popular test simulators.

Methods. In the MoG benchmark, we compare R2OMC with a straightforward use of the ROMC framework (NAIVE-ROMC) and two neural methods: NPE (specifically, "SNPE-C" by Greenberg et al., 2019) and NLE (specifically, "NLE-A" by Papamakarios et al., 2019). For the MoG benchmark, the maximal simulation budget is 10^4. This means 10^4 simulated samples for NPE and NLE , and maximally 10^4 simulator calls for R2OMC and NAIVE-ROMC. For the SBI benchmark, we test multiple budgets. Both R2OMC and NAIVE-ROMC use the squared Euclidean norm as distance function. NAIVE-ROMC does not filter out distractors and concatenates iid observations to form a single vector of observed data. We use our `JAX` implementation for R2OMC, and also a `JAX` implementation for NAIVE-ROMC, which gives both of them access to gradients, and the `SBI Pytorch` implementation for NLE and NPE (Tejero-Cantero et al., 2020). In the SBI benchmark, we compare R2OMC with Rejection ABC (REJ-ABC), Sequential Monte Carlo ABC (SMC-ABC), Neural Likelihood Estimation (NLE), Neural Ratio Estimation (NRE), and their sequential variants. All experiments were conducted on a single Dell XPS PC equipped with an Intel® Core™ i7-10750H CPU @ 2.60GHz with 12 cores. Appendix C contains further details on the setup and methods.

Metrics. We evaluate all the methods with the `C2ST` score, which uses classification to measure the similarity between two data sets (Gutmann et al., 2018; Lueckmann et al., 2021). The first set are the samples from the approximate posterior while the second set are the ground truth posterior samples. The best score is 0.5, indicating that the two sets are indistinguishable, while the worst score is 1.0, indicating perfect separation. As classifier, we use a fully-connected neural network trained on 1,000 samples from each set and report the mean `C2ST` score over 5 independent runs.

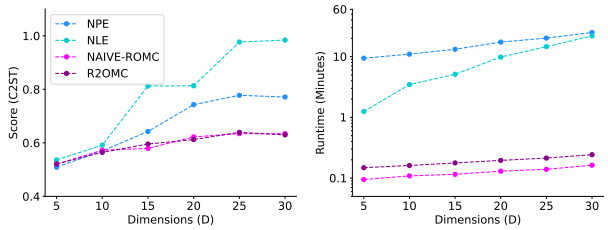
4.1 MoG benchmark

The simulator models used here allow us to systematically evaluate SBI methods when (a) the dimensionality of the parameter space D, (b) the number of observations N, and (c) the number of uninformative dimensions increases, without altering the problem's structure. Moreover, they are amenable to closed form approximations to the posterior. For all simulators, we use a uniform prior $p(\boldsymbol{\theta}) = \mathcal{U}(-\mathbf{15}, \mathbf{15})$.

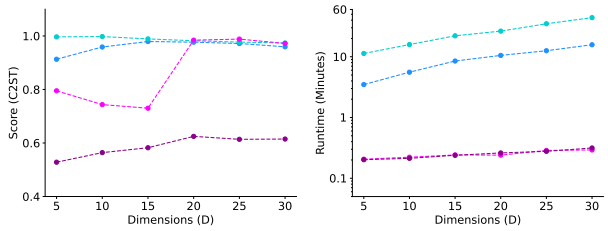
The simulators are

- $S_{\texttt{base}} : \mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2\mathbf{I})$,
- $S_{\texttt{MoG}} : \mathbf{y} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2\mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2\mathbf{I})\right)$,

for more, see Appendix D. Each simulator S is used in a simple and extended setup. In the simple setup, the output has the same dimensionality as the parameter vector ($D_y = D$) and we have a single observation $\mathbf{y}^o \in \mathbb{R}^{D_y}$. In the extended setup, we further consider distractors and/or additional iid data points. In the distractors version, we denote the simulator as $S^{\texttt{dist}}$ and we append 100 distractors to the simulator output, thus augmenting the output space to $D_y + 100$. The distractor variables are sampled from $\mathbf{y}^{\texttt{dist}} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I})$,

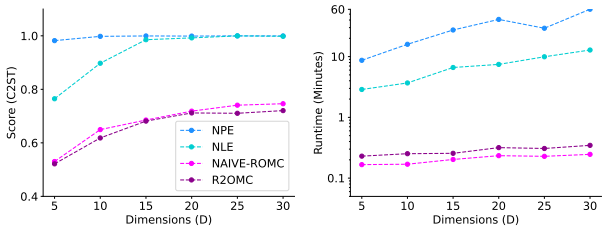


(a) $S_{\texttt{base}}$: C2ST (b) $S_{\texttt{base}}$: Log runtime

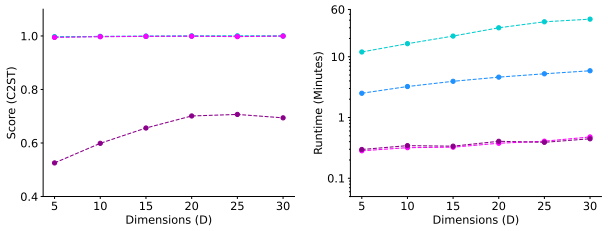


(c) $S^{\texttt{dist}}_{\texttt{base}}$: C2ST (d) $S^{\texttt{dist}}_{\texttt{base}}$: Log runtime

Figure 3: Base simulator, varying dimensionality.



(a) $S_{\texttt{MoG}}$: C2ST (b) $S_{\texttt{MoG}}$: Log runtime



(c) $S^{\texttt{dist}}_{\texttt{MoG}}$: C2ST (d) $S^{\texttt{dist}}_{\texttt{MoG}}$: Log runtime

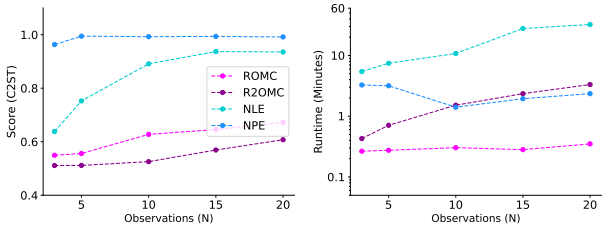
Figure 4: MoG simulator, varying dimensionality.

with $\boldsymbol{\mu} \sim \mathcal{U}(-10, 10)$. In the version with multiple iid observations, we denote the simulator as $S^{\texttt{multi}}$.

As observed data, we use $\mathbf{y}_n^o \approx (5, \dots, 5)$, that is a vector of 5's with some small noise around it. In case of distractors, we append 100 zeros to it. The posterior for $S_{\texttt{linear}}$ is then approximated by a truncated Gaussian, while the posterior for $S_{\texttt{MoG}}$ is well approximated by a truncated bi-modal Gaussian, see Appendix C.

High-dimensional simulators. We set $N = 1$ and vary the dimensionality of the parameter space, using $D \in \{5, 10, 25, 30\}$. Figure 3 shows the results for $S_{\texttt{base}}$ with $\sigma = 1.5$. In case of no distractors, all methods perform similarly for $D < 10$ in terms of accuracy but the neural methods start to struggle as D increases. Both R2OMC and NAIVE-ROMC perform well as they both use gradients, enabling scalability. When distractors are added, however, all methods but R2OMC fail already at low dimensions. In terms of compute time, the ROMC-based methods are at least an order of magnitude faster. In particular note that the runtime is only weakly affected by the dimensionality. Figure 4 shows the results for $S_{\texttt{MoG}}$ with $\sigma_1 = 0.1$ and $\sigma_2 = 1$. The results are largely similar except that the difference in performance is notable at low dimensions already.

Multiple observations. We set $D = 10$ and vary the number N of iid observations in $\{2, 5, 10, 15, 20\}$. To isolate the effect of the number of observations, we do not consider distractors. The results are shown in Figure 5 for $S_{\texttt{base}}$ ($\sigma = 1$) and Figure 6 for $S_{\texttt{MoG}}$ ($\sigma_1 = \sigma_2 = 1$). NPE is strongly affected by multiple observations, breaking down quickly. NLE performs better but the performance deteriorates significantly as



(a) $S_{\texttt{base}}$: C2ST (b) $S_{\texttt{base}}$: Log runtime

Figure 5: Base simulator, varying nb of observations.

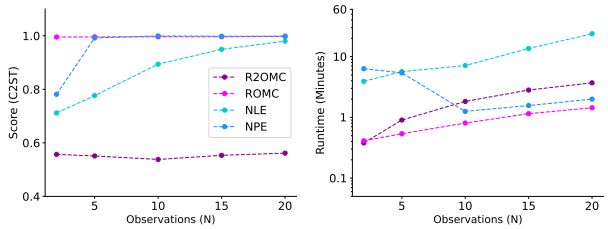
N increases. While NAIVE-ROMC performs well for $S_{\texttt{base}}$, it breaks for $S_{\texttt{MoG}}$. In contrast, R2OMC achieves good performance across all N. In line with the analysis in Section 3.3, the runtime increases roughly linearly with N (logarithmically on the log-scale).

Summary. Although the used simulators are simple, we observe that most methods struggle when the parameter dimensionality increases, multiple observations are used, or distractors (uninformative dimensions) are present, in line with Figure **??**. R2OMC, in contrast, performs well in all-setups (`C2ST` between 0.5 and 0.7) while running at least an order of magnitude faster than the tested neural methods.

4.2 SBI benchmark

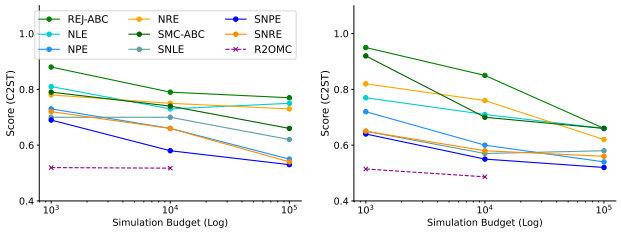
The SBI Benchmark (Lueckmann et al., 2021) enables the comparison with a range of methods. We select the following two experiments from the benchmark:

Gaussian Mixture (T.7). The simulator is $\mathbf{y}|\boldsymbol{\theta} \sim 0.5\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + 0.5\mathcal{N}(\boldsymbol{\theta}, 0.01\mathbf{I})$. The parameters $\boldsymbol{\theta}$ are two-dimensional and the prior is $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$.



(a) $S_{\texttt{MoG}}$: C2ST (b) $S_{\texttt{MoG}}$: Log runtime

Figure 6: MoG simulator, varying nb of observations.



(a) Mixture of Gaussians (b) 2-moons

Figure 7: SBI benchmark: performance (C2ST) vs compute budget (log number of simulator runs).

The ground truth posterior is $\boldsymbol{\theta}|\mathbf{y}^o \sim 0.5\mathcal{N}(\mathbf{y}^o, \mathbf{I}) + 0.5\mathcal{N}(\mathbf{y}^o, 0.01\mathbf{I})$, truncated to the support of the prior. The task tests whether methods can accurately estimate a mixture of two Gaussians with same mean but one with much broader variance than the other.

2-moons (T.8). The simulator is $\mathbf{y}|\boldsymbol{\theta} \sim (r\cos(\alpha) + 0.25, r\sin(\alpha)) + (-|\theta_1 + \theta_2|/\sqrt{2}, (-\theta_1 + \theta_2)/\sqrt{2})$, $\alpha \sim \mathcal{U}(-\pi/2, \pi/2)$, $r \sim \mathcal{N}(0.1, 0.01^2)$. The parameters $\boldsymbol{\theta} = (\theta_1, \theta_2)$ are two-dimensional and the prior is $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{1}, \mathbf{1})$. This creates a posterior with two half-moon shape modes. The task tests whether the methods can accurately estimate a posterior with both global (bimodality) and local (crescent shape) structure.

Results. The results are shown in Figure 7, plotting the performance in terms of `C2ST` score versus the number of simulator runs needed. The figure shows that R2OMC has a `C2ST` score close to 0.5 even with a simulation budget of 10^3 samples, while the rest of the methods need a budget of 10^5 or more to achieve a similar score and perform much worse for smaller budgets. As in the Gaussian benchmark, we observe that most SBI methods fail to produce accurate posterior samples without a large simulation budget even in relatively simple tasks.

4.3 Image Example

5 Conclusion

We have presented R2OMC, a new Bayesian inference method for simulator models that addresses three key scalability challenges in simulation-based (likelihood-free) inference, namely high-dimensional parameter spaces, multiple observations, and the possible presence of uninformative dimensions (i.e., distractors). R2OMC builds on the ROMC framework and formulates the inference problem in terms of a set of deterministic optimization problems that can be solved with gradient-descent, enabling inference in high-dimensional parameter spaces. It further introduces a gradient-based filtering step to mask distractors during inference. It uses a novel adaptive importance sampling approach that efficiently identifies posterior samples that explain *all* available iid observations. We offer an efficient `JAX`-based implementation that leverages key auto-differentiation and vectorization features to accelerate the inference. Simulations performed with novel experimental settings as well as the comprehensive SBI benchmark show that R2OMC achieves improved accuracy (close to the optimal 0.5 `C2ST`) compared to existing methods, while using a smaller number of simulator calls and a fraction of the computational time.

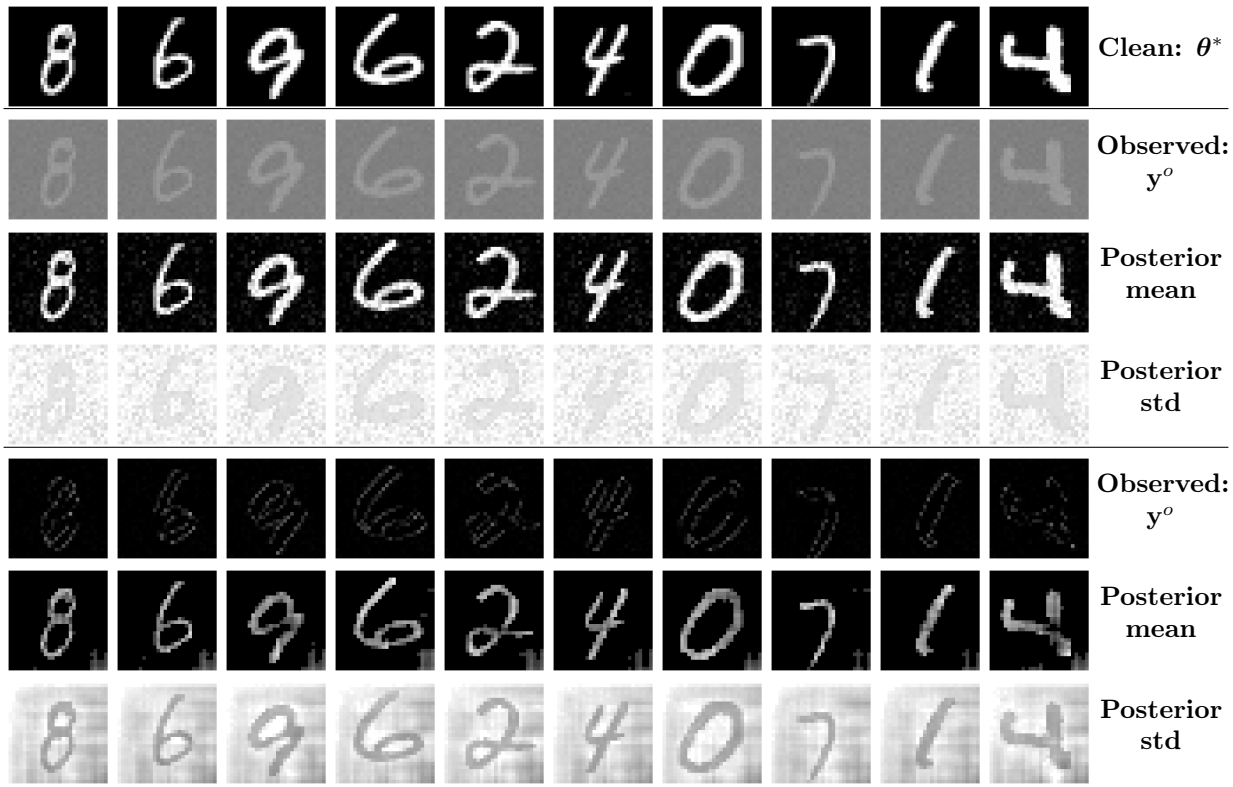


Figure 8: Your figure caption here.

References

Jarno Lintusaari, Michael U Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Fundamentals and recent developments in approximate bayesian computation. *Systematic biology*, 66(1): e66–e82, 2017.

Scott A Sisson, Yanan Fan, and Mark Beaumont. *Handbook of approximate Bayesian computation*. CRC press, 2018.

Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48): 30055–30062, 2020.

Jean-Michel Marin, Pierre Pudlo, Christian P. Robert, and Robin J. Ryder. Approximate bayesian computational methods. *Statistics and Computing*, 22(6):1167–1180, 2012. ISSN 1573-1375. doi:10.1007/s11222-011-9288-2. URL https://doi.org/10.1007/s11222-011-9288-2.

Y. Chen, D. Zhang, M. U. Gutmann, A. Courville, and Z. Zhu. Neural approximate sufficient statistics for implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=SRDuJssQud.

Yanzhi Chen, Michael U. Gutmann, and Adrian Weller. Is learning summary statistics necessary for likelihood-free inference? In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 4529–4544. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/chen23h.html.

Simon N Wood. Statistical inference for noisy nonlinear ecological dynamic systems. *Nature*, 466(7310):1102–1104, 2010.

Leah F Price, Christopher C Drovandi, Anthony Lee, and David J Nott. Bayesian synthetic likelihood. *Journal of Computational and Graphical Statistics*, 27(1):1–11, 2018.

Owen Thomas, Ritabrata Dutta, Jukka Corander, Samuel Kaski, and Michael U Gutmann. Likelihood-free inference by ratio estimation. *Bayesian Analysis*, 17(1):1–31, 2022.

Conor Durkan, Iain Murray, and George Papamakarios. On contrastive learning for likelihood-free inference. In *International conference on machine learning*, pages 2771–2781. PMLR, 2020.

George Papamakarios, David Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd international conference on artificial intelligence and statistics*, pages 837–848. PMLR, 2019.

David Greenberg, Marcel Nonnenmacher, and Jakob Macke. Automatic posterior transformation for likelihood-free inference. In *International Conference on Machine Learning*, pages 2404–2414. PMLR, 2019.

George Papamakarios and Iain Murray. Fast ε-free inference of simulation models with bayesian conditional density estimation. *Advances in neural information processing systems*, 29, 2016.

A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, and S. Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008.

Gloria M. Monsalve-Bravo, Brodie A. J. Lawson, Christopher Drovandi, Kevin Burrage, Kevin S. Brown, Christopher M. Baker, Sarah A. Vollert, Kerrie Mengersen, Eve McDonald-Madden, and Matthew P. Adams. Analysis of sloppiness in model simulations: Unveiling parameter uncertainty when mathematical models are fitted to data. 8(38): eabm5952, 2022. doi:10.1126/sciadv.abm5952. URL https://www.science.org/doi/abs/10.1126/sciadv.abm5952.

Jan-Matthis Lueckmann, Jan Boelts, David Greenberg, Pedro Goncalves, and Jakob Macke. Benchmarking

simulation-based inference. In *International conference on artificial intelligence and statistics*, pages 343–351. PMLR, 2021.

Ted Meeds and Max Welling. Optimization monte carlo: Efficient and embarrassingly parallel likelihood-free inference. *Advances in Neural Information Processing Systems*, 28, 2015.

Borislav Ikonomov and Michael U Gutmann. Robust optimisation monte carlo. In *International Conference on Artificial Intelligence and Statistics*, pages 2819–2829. PMLR, 2020.

Vasilis Gkolemis, Michael Gutmann, and Henri Pesonen. An extendable python implementation of robust optimisation monte carlo. *arXiv preprint arXiv:2309.10612*, 2023.

Alvaro Tejero-Cantero, Jan Boelts, Michael Deistler, Jan-Matthis Lueckmann, Conor Durkan, Pedro J. Gonçalves, David S. Greenberg, and Jakob H. Macke. sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, 5(52):2505, 2020. doi:10.21105/joss.02505. URL https://doi.org/10.21105/joss.02505.

Michael U Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Likelihood-free inference via classification. *Statistics and Computing*, 28:411–425, 2018.

Jarno Lintusaari, Henri Vuollekoski, Antti Kangasraasio, Kusti Skytén, Marko Jarvenpaa, Pekka Marttinen, Michael U Gutmann, Aki Vehtari, Jukka Corander, and Samuel Kaski. Elfi: Engine for likelihood-free inference. *Journal of Machine Learning Research*, 19(16):1–7, 2018.

Checklist

The checklist follows the references. For each question, choose your answer from the three possible options: Yes, No, Not Applicable. You are encouraged to include a justification to your answer, either by referencing the appropriate section of your paper or providing a brief inline description (1-2 sentences). Please do not modify the questions. Note that the Checklist section does not count towards the page limit. Not including the checklist in the first submission won't result in desk rejection, although in such case we will ask you to upload it during the author response period and include it in camera ready (if accepted).

In your paper, please delete this instructions block and only keep the Checklist section heading above along with the questions/answers below.

1. For all models and algorithms presented, check if you include:
   (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes]
   (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes]
   (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No]

2. For any theoretical claim, check if you include:
   (a) Statements of the full set of assumptions of all theoretical results. [Not Applicable]
   (b) Complete proofs of all theoretical results. [Not Applicable]
   (c) Clear explanations of any assumptions. [Not Applicable]

3. For all figures and tables that present empirical results, check if you include:
   (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No] Not now, but code will be made available if accepted.
   (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes]
   (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes]
   (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [Yes]

4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
   (a) Citations of the creator If your work uses existing assets. [Yes]
   (b) The license information of the assets, if applicable. [Yes]
   (c) New assets either in the supplemental material or as a URL, if applicable. [Not Applicable]
   (d) Information about consent from data providers/curators. [Not Applicable]
   (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable]

5. If you used crowdsourcing or conducted research with human subjects, check if you include:

(a) The full text of instructions given to participants and screenshots. [Not Applicable]

(b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable]

(c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable]

A Methods

Throughout the paper we run experiments based on four methods: R2OMC (our proposal), NAIVE-ROMC (a straightforward use of the ROMC framework by Ikonomov and Gutmann, 2020), Neural Posterior Estimation (NPE) (Greenberg et al., 2019), Neural Likelihood Estimation (NLE) (Papamakarios et al., 2019).

R2OMC. A detailed description of the method can be found in Algorithm 1.

In Step 2 of Algorithm 1, i.e., computing the uninformative dimensions, we typically use around 50 samples from the distribution $p(u)$ and another 50 samples from $p(\boldsymbol{\theta})$, with the parameter τ set to machine precision, i.e., $\tau \approx 0$. Similar results can be obtained with fewer samples, such as 10 or 20.

For Step 3, i.e., computing the optimal points $\boldsymbol{\theta}^*_{i,n}$, we use the Adam optimizer, with a learning rate between 0.01 and 0.2, depending on the specific problem. Our default choice is 0.1. The number of optimization steps varies between 100 and 400, with 50 as the default. The distance function $d(\boldsymbol{\theta})$ is defined as the squared Euclidean distance between the simulator output and the observation, i.e., $d(\boldsymbol{\theta}) = \|\mathbf{y} - \mathbf{y}^o_n\|_2^2$. The number of seeds S used ranges from 500 to 5000, with 1000 as the default. The number of seeds corresponds to the simulation budget; for example, if a budget of 10,000 runs is available, we use (at most) 10,000 seeds. Typically, we select 80% of the seeds based on the best $d^n_i(\boldsymbol{\theta}^*_{i,n})$. However, the percentage of accepted seeds can vary between 50% and 100%, depending on the absolute values of $d^n_i(\boldsymbol{\theta}^*_{i,n})$ for $i = 1, \ldots, S$.

For Step 5, i.e., constructing the hyperboxes, we use Algorithm 2. We normally set ϵ to be twice the distance of the 'worst' accepted seed, i.e., $\epsilon = 2 \max_i d^n_i(\boldsymbol{\theta}^*_{i,n})$, where i is an index over the accepted seeds. Alternatively, ϵ can be set to a fixed value such as 0.1. The typical settings for step size, number of steps, and refinements are $\eta = 0.1$, $L = 100$, and $R = 1$, respectively.

For Step 7, i.e., sampling from the proposal distribution, we generate between 1000 and 100,000 candidate samples, with the default being 5000. From these candidates, we select 1000 via weighted sampling. For computing the weights, $\{w_p\}_{p=1}^P$, we set ϵ to a value about twice the distance of the 'worst' accepted seed, i.e., $\epsilon = 2 \max_{i,n} d^n_i(\boldsymbol{\theta}^*_{i,n})$. However, this can vary depending on the problem, and by testing different values.

Finally, we keep 1000 posterior samples, which are used to compute the `C2ST` score using another 1000 samples drawn from the true posterior.

Algorithm 2 Hyperbox Computation

Input: Optimal point $\boldsymbol{\theta}^*$, distance function $d(\boldsymbol{\theta})$, Jacobian $\mathbf{J}$ at $\boldsymbol{\theta}^*$

Parameters: Step size η, number of refinements R, number of steps L, distance threshold ϵ

Compute eigenvectors $\mathbf{v}_d$ of $\mathbf{J}^T\mathbf{J}$ $(d = 1, \ldots, D)$
for $d = 1$ to D **do**
  Initialize endpoint $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta}^*$
  Set refinement counter $r \leftarrow 0$
  repeat
    Set step counter $l \leftarrow 0$
    repeat
      Increment step counter $l \leftarrow l + 1$
      Update $\tilde{\boldsymbol{\theta}} \leftarrow \tilde{\boldsymbol{\theta}} + \eta \mathbf{v}_d$
    until $d(\boldsymbol{\theta}) > \epsilon$ or $l = L$
    Move back one step $\tilde{\boldsymbol{\theta}} \leftarrow \tilde{\boldsymbol{\theta}} - \eta \mathbf{v}_d$
    Halve the step size $\eta \leftarrow \eta/2$
    Increment refinement counter $r \leftarrow r + 1$
  until $r = R$
  Set $\tilde{\boldsymbol{\theta}}$ as the endpoint
  Repeat steps 3-15 for $\mathbf{v}_d = -\mathbf{v}_d$ and set $\tilde{\boldsymbol{\theta}}$ as the negative endpoint
end for
Define a uniform distribution q over the hyperbox
return q

NAIVE-ROMC. We use the Algorithm presented as ROMC in Ikonomov and Gutmann (2020). The characterization NAIVE indicates the use of a naive distance function in case of multiple observations. As described in the main paper, in case of N observations, we concatenate them in a single vector $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \dots, \mathbf{y}^{o,(N)})$, then, run the simulator N times, concatenate the ouputs, $\mathbf{y} = (\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)})$ and set the distance to be the squared Euclidean distance between the two vectors, $d(\boldsymbol{\theta}) = ||\mathbf{y} - \mathbf{y}^o||_2^2$. Although using this approach for defining $d(\boldsymbol{\theta})$ has disadvantages, that is why we name it 'NAIVE', there is no other (simple) approach that leads to a better posterior approximation, under multiple observations.

NAIVE-ROMC is practically the same as R2OMC (Algorithm 1 of the main text), if (a) we omit Step 2, i.e., computing the uninformative dimensions and (b) set $N = 1$. This is why NAIVE-ROMC and R2OMC have a similar `C2ST` score in problems without distractors and with a single observation. In case of multiple observations, we use as $\mathbf{y}$ and $\mathbf{y}^o$, the concatenated vectors as described above. For NAIVE-ROMC, we use a `JAX`-based implementation that is similar to the one we use for R2OMC. The reason for not using its existing implementation (Gkolemis et al., 2023) of the ELFI package (Lintusaari et al., 2018) is that it does not use gradients, so it cannot scale to high dimensions. The hyperparameters used in NAIVE-ROMC are similar to R2OMC, as described above.

NPE As Neural Posterior Estimation (NPE) we use the one denoted as as SNPE_C in the SBI benchmark (Lueckmann et al., 2021), which is a method proposed by Greenberg et al. (2019). SNPE_C estimates the posterior by training a neural network $F(\mathbf{y}, \phi)$ to approximate $p(\boldsymbol{\theta}|\mathbf{y})$ using a density estimator $q_F(\mathbf{y}, \phi)(\boldsymbol{\theta})$. For further details, see (Greenberg et al., 2019).

We use the `Pytorch` implementation provided by the SBI package, using mostly the default parameters. As density estimator we use the a neural network with 50 hidden units, `sbi.utils.get_nn_models.posterior_nn()` with parameters `model='nsf'`, `hidden_features=50`, `z_score_x='independent'`, and `z_score_x='independent'`. During training, we use 10 atoms, a batch size of 1000 (or the whole dataset) and a maximum number of 500 epochs.

We use a simulation budget of $10,000$ simulator executions, which means that we train the density estimator on $10,000$ samples. We normally use 2 training rounds, each with 5000 training samples: the first set of 5000 is drawn from the prior, while the second set is sampled from the approximate posterior learned after the first round. Occasionally, if the initial 5000 samples do not yield a reliable approximation and conditioning on $\mathbf{y}^o$ leads to instabilities, we use a single round of 10000 samples.

In cases with multiple observations, since NPE cannot handle them separately, we follow the same approach as in NAIVE-ROMC(see above); we concatenate the observations and the outputs to one vector each.

NLE As NLE we use the method proposed by Papamakarios et al. (2019). NLE approximates the likelihood $p(\mathbf{y}^o|\theta)$ using neural density estimation. The process involves generating a dataset $D = \{(\boldsymbol{\theta}_k, \mathbf{y}_k)\}_{k=1}^K$ by sampling $\boldsymbol{\theta}_k \sim p(\boldsymbol{\theta})$ and simulating $\mathbf{y}_k \sim p(\mathbf{y}|\boldsymbol{\theta}_k)$. A conditional neural density estimator $q_\psi(\mathbf{y}|\boldsymbol{\theta})$ is then trained to model the likelihood.

Once trained, samples from the posterior $\hat{p}(\boldsymbol{\theta}|\mathbf{y}^o) \propto q_\psi(\mathbf{y}^o|\boldsymbol{\theta})p(\boldsymbol{\theta})$ are obtained using MCMC. For density estimation, we use the default SBI settings, i.e., a Masked Autoregressive Flow (MAF) with five flow transforms and 50 hidden units per block. For MCMC, 250 warm-up steps and posterior samples are obtained with thinning.

We use a simulation budget of $10,000$ simulator executions, which means that we train the density estimator on $10,000$ samples. We always use a single training round. NLE can handle multiple observations; we simply pass a `numpy` array of shape: `(N,D)`, where N is the number of observations and D their dimensionality. Internally, NLE evaluates each observation using the neural density estimator $q_\psi(\mathbf{y}|\boldsymbol{\theta})$ and multiplies them to estimate the joint likelihood and then uses MCMC to sample from the posterior.

B Introductory Example

Notation. We use $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \Sigma)$ to denote the probability density function (pdf) of a Gaussian random variable $\mathbf{y}$, while $\mathcal{N}(\boldsymbol{\theta}, \Sigma)$ denotes the random variable itself. Typically, $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \Sigma)$ is used when deriving expressions, whereas $\mathcal{N}(\boldsymbol{\theta}, \Sigma)$ is used to indicate sampling, such as in $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \Sigma)$. We use $\mathcal{U}(\mathbf{a}, \mathbf{b})$ to denote a multivariate random variable that is uniformly distributed on the hypercube defined by vector $\mathbf{a}$ with the lower boundaries and the vector $\mathbf{b}$ with the upper boundaries. The dimensions are typically clear from the context if not explicitly stated. Bold indicates a vector, e.g., $\boldsymbol{\theta} \in \mathbb{R}^D$.

Setup. We provide further details about the introductory example shown in Figure **??**. The introductory example consists of five different versions: (1) Simple, (2) High-dimensional, (3) Distractor, (4) Multiple observations, and (5) Combined.

The simulator of the Simple (1), High-dimensional (2) and Multiple observations (4) versions is:

$$\mathbf{y} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I})\right) \tag{15}$$

At the Distractor (3) and the Combined (5) versions, we add $D_{\text{dist}} = 10$ distractor dimensions to the output, so the simulator becomes:

$$(\mathbf{y}^{(1)}, \mathbf{y}^{(2)}), \quad \mathbf{y}^{(1)} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I})\right), \quad \mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I}), \tag{16}$$

where $\boldsymbol{\mu} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$. In all cases the prior is given by $p(\boldsymbol{\theta}) = \mathcal{U}(-\mathbf{15}, \mathbf{15})$.

B.1 Simple Case

The simulator is the one in (15) with $D = 5$ and the observation is $\mathbf{y}^o = (5, \ldots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^5$. The ground truth posterior for $\mathbf{y}^o = (5, \ldots, 5)$ is given by:

$$\begin{aligned}
p(\boldsymbol{\theta}|\mathbf{y}^o) &\propto p(\boldsymbol{\theta}) p(\mathbf{y}^o|\boldsymbol{\theta}) && (17)\\
&\propto p(\boldsymbol{\theta})(\mathcal{N}(\mathbf{y}^o; \boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(\mathbf{y}^o; -\boldsymbol{\theta}, \mathbf{I})) && (18)\\
&\propto p(\boldsymbol{\theta})(\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}^o, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}^o, \mathbf{I})) && (19)\\
&\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^5, \\ 0, & \text{otherwise.} \end{cases} && (20)
\end{aligned}$$

From (18) to (19), we apply the property: $\mathcal{N}(\mathbf{y}; -\boldsymbol{\theta}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}, \Sigma)$.

B.2 High-dimensional Case

The simulator is the one in (15) with $D = 20$ and the observation is $\mathbf{y}^o = (5, \ldots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^{20}$. The derivation of (20) holds irrespectively of the dimensionality, so the ground truth posterior for $\mathbf{y}^o = (5, \ldots, 5)$ is:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^{20}, \\ 0, & \text{otherwise.} \end{cases} \tag{21}$$

B.3 Distractor Case

The simulator is the one in (16) with $D = 5$ and $D_{\text{dist}} = 10$, leading to an output dimensionality of $D_y + D_{\text{dist}} = 15$. The observation is $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})$, where $\mathbf{y}^{o,(1)} = (5, \ldots, 5) + \epsilon$ and $\mathbf{y}^{o,(2)} = (0, \ldots, 0) + \epsilon$. Below, we show that the ground truth posterior is equivalent to the simple case (20), as the distractor dimensions do not affect

the posterior:

$$
\begin{aligned}
p(\boldsymbol{\theta}|\mathbf{y}^o) &\propto p(\boldsymbol{\theta})p(\mathbf{y}^o|\boldsymbol{\theta}) && (22)\\
&\propto p(\boldsymbol{\theta})p((\mathbf{y}^{o,(1)},\mathbf{y}^{o,(2)})|\boldsymbol{\theta}) && (23)\\
&\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}|\boldsymbol{\theta}) && (24)\\
&\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}) && (25)\\
&\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) && (26)\\
&\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta};\mathbf{5},\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\mathbf{I}), & \text{if } \boldsymbol{\theta}\in[-15,15]^5,\\ 0, & \text{otherwise.}\end{cases} && (27)
\end{aligned}
$$

From (23) to (24), we use that $\mathbf{y}^{o,(1)}$ is independent of $\mathbf{y}^{o,(2)}$ and from (24) to (25), that $\mathbf{y}^{o,(2)}$ is independent of $\boldsymbol{\theta}$.

B.4 Multiple Observations Case

The simulator is the one in (15) with $D=5$ and the set of $N=4$ iid observations is $\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o = (5,\dots,5)+\epsilon$ for $n=1,2$ and $\mathbf{y}_n^o=(-5,\dots,-5)+\epsilon$ for $n=3,4$. We will derive the ground truth posterior for the general case of N observations, where $\mathbf{y}_n^o=(5,\dots,5)+\epsilon$ for $n\in\{1,\dots,N/2\}$ and $\mathbf{y}_n^o=(-5,\dots,-5)+\epsilon$ for $n\in\{N/2+1,\dots,N\}$.

$$
\begin{aligned}
p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) &\propto p(\boldsymbol{\theta})\prod_n^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) && (28)\\
&\propto p(\boldsymbol{\theta})\prod_{n=1}^{N/2} p(\mathbf{y}_n^o|\boldsymbol{\theta})\prod_{n=N/2+1}^{N} p(\mathbf{y}_n^o|\boldsymbol{\theta}) && (29)\\
&\propto p(\boldsymbol{\theta})\prod_{n=1}^{N/2} (\mathcal{N}(\mathbf{5};\boldsymbol{\theta},\mathbf{I}) + \mathcal{N}(-\mathbf{5};\boldsymbol{\theta},\mathbf{I}))\prod_{n=N/2+1}^{N} (\mathcal{N}(-\mathbf{5};\boldsymbol{\theta},\mathbf{I}) + \mathcal{N}(\mathbf{5};\boldsymbol{\theta},\mathbf{I})) && (30)\\
&\propto p(\boldsymbol{\theta})\prod_{n=1}^{N} (\mathcal{N}(\mathbf{5};\boldsymbol{\theta},\mathbf{I}) + \mathcal{N}(-\mathbf{5};\boldsymbol{\theta},\mathbf{I})) && (31)\\
&\propto p(\boldsymbol{\theta})\prod_{n=1}^{N} (\mathcal{N}(\boldsymbol{\theta};\mathbf{5},\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\mathbf{I})) && (32)\\
&\propto\!\!\!\sim p(\boldsymbol{\theta})(\mathcal{N}(\boldsymbol{\theta};\mathbf{5},\mathbf{I}))^N + (\mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\mathbf{I}))^N && (33)\\
&\propto\!\!\!\sim p(\boldsymbol{\theta})(\mathcal{N}(\boldsymbol{\theta};\mathbf{5},\frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\frac{1}{N}\mathbf{I})) && (34)\\
&\propto\!\!\!\sim \begin{cases} \mathcal{N}(\boldsymbol{\theta};\mathbf{5},\frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta}\in[-15,15]^5,\\ 0, & \text{otherwise.}\end{cases} && (35)
\end{aligned}
$$

From (29) to (30), we simply plug in the corresponding observations. From (31) to (32), we apply the property: $\mathcal{N}(\mathbf{y};-\boldsymbol{\theta},\Sigma)=\mathcal{N}(\boldsymbol{\theta};-\mathbf{y},\Sigma)$. From (32) to (33), we use that the terms that include the product $\mathcal{N}(\boldsymbol{\theta};\mathbf{5},\mathbf{I})\mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\mathbf{I})$ have a much smaller magnitude than the two terms $\mathcal{N}(\boldsymbol{\theta};\mathbf{5},\mathbf{I})^N$ and $\mathcal{N}(\boldsymbol{\theta};-\mathbf{5},\mathbf{I})^N$. Therefore, we keep only the latter in (33). From (33) to (34), we use the property: $\mathcal{N}(\boldsymbol{\theta};\boldsymbol{\alpha},\Sigma)^N=\mathcal{N}(\boldsymbol{\theta};\boldsymbol{\alpha},\frac{1}{N}\Sigma)$.

B.5 Combined Case

The simulator is the one in (16) with $D=20$ and $D_{\text{dist}}=10$, leading to an output dimensionality of $D_y+D_{\text{dist}}=30$. The set of $N=4$ iid observations is $\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o=(\mathbf{y}^{o,(1)},\mathbf{y}^{o,(2)})$, with $\mathbf{y}^{o,(1)}$ equal $(5,\dots,5)+\epsilon$ for $n\in\{1,\dots,N/2\}$ and $(-5,\dots,-5)+\epsilon$ for $n\in\{N/2+1,\dots,N\}$ and $\mathbf{y}^{o,(2)}$ equal $(0,\dots,0)+\epsilon$ for all n.

Using the properties of the previous cases we obtain the ground truth posterior, which is the same as in the

multiple observations case (35) for $D = 20$:

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto p(\boldsymbol{\theta}) \prod_n^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) \tag{36}$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p((\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})|\boldsymbol{\theta}) \tag{37}$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) p(\mathbf{y}^{o,(2)}|\boldsymbol{\theta}) \tag{38}$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) p(\mathbf{y}^{o,(2)}) \tag{39}$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) \tag{40}$$

From (37) to (38), we use that the observations are independent and from (38) to (39) that $\mathbf{y}^{o,(2)}$ is independent of $\boldsymbol{\theta}$. Using the derivation of the multiple observations case (35) gives

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto\!\!\!\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^{20}, \\ 0, & \text{otherwise.} \end{cases} \tag{41}$$

B.6 Results

Setup. For NAIVE-ROMCand R2OMC, a maximum of $S = 5000$ seeds was used, corresponding to approximately 5000 independent simulator calls, which is (approximately) a simulation budget of 5000. NLE and NPE were trained with a simulation budget of 10000 samples. For NPE, we used $R = 2$ rounds with 5000 samples each, while for NLE, we used a single round of 10000 samples. The default settings from the SBI library Lueckmann et al. (2021) were utilized for both, with a training batch size of 1000 and a maximum of 300 training epochs.

Conclusions. The results of the introductory example are presented in Figure **??** in the main text. In the simple scenario, all methods provide a reasonably accurate approximation of the ground truth posterior. In the high-dimensional case, both NPE and NLE encounter difficulties, indicating that high-dimensional problems are challenging for these methods even when the simulator is relatively simple. In contrast, NAIVE-ROMC and R2OMC yield a good approximation of the ground truth posterior, demonstrating their robustness in high-dimensional settings. Note that in the simple scenario, NAIVE-ROMC and R2OMC are equivalent, since there are no distractors and only one observation. In the distractors scenario, NPE struggles, NLE is more robust than NPE, but still faces challenges in providing an accurate approximation, while NAIVE-ROMC struggles. R2OMC provides a good approximation of the ground truth posterior. In the multiple observations scenario, only NLE and R2OMC succeed in generating a reliable approximation of the ground truth posterior. Finally, in the combined scenario, only R2OMC delivers a satisfactory approximation of the ground truth posterior.

C Additional information on the experiments

We here provide some additional information on the experiments presented in the main text.

C.1 MoG Benchmark - High Dimensional Simulators

C.1.1 Base simulator - without distractors

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$

Simulator $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{I})$, with $\sigma^2 = 1.5^2$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^D$

Observation $\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^D$

Posterior $p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

Proof of the posterior:

$$
\begin{aligned}
p(\boldsymbol{\theta}|\mathbf{y}^o) &\propto p(\boldsymbol{\theta}) p(\mathbf{y}^o|\boldsymbol{\theta}) && (42)\\
&\propto p(\boldsymbol{\theta}) \mathcal{N}(\mathbf{y}^o; \boldsymbol{\theta}, \sigma^2 \mathbf{I}) && (43)\\
&\propto p(\boldsymbol{\theta}) \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}^o, \sigma^2 \mathbf{I}) && (44)\\
&\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} && (45)
\end{aligned}
$$

From (43) to (44), we apply the property: $\mathcal{N}(\mathbf{y}; -\boldsymbol{\theta}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}, \Sigma)$.

C.1.2 Base simulator - with distractors

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$

Simulator $\mathbf{y} = (\mathbf{y^{(1)}}, \mathbf{y^{(2)}})$,
$\mathbf{y^{(1)}} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{I})$, $\sigma^2 = 1.5^2$,
$\mathbf{y^{(2)}} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I})$, $\boldsymbol{\mu} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^{D+100}$, $\mathbf{y}^{(1)} \in \mathbb{R}^D$, $\mathbf{y}^{(2)} \in \mathbb{R}^{100}$

Observation $\mathbf{y}^o = (\mathbf{y^{o,(1)}}, \mathbf{y^{o,(2)}})$, where $\mathbf{y^{o,(1)}} = (5, \dots, 5) + \epsilon$ and $\mathbf{y^{o,(2)}} = (0, \dots, 0) + \epsilon$

Posterior $p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The ground truth posterior is the same as in (45), as the distractors do not affect the posterior, see (27).

C.1.3 MoG simulator - without distractors

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$

Simulator $\mathbf{y} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2 \mathbf{I})\right)$, $\sigma_1^2 = 0.1$, $\sigma_2^2 = 1$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^D$

Observation $\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^D$

Posterior $p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \sigma_2^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The MoG simulator is the same as in the introductory example, so the proof is equivalent to Section B.

C.1.4 MoG simulator - with distractors

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$
Simulator	$\mathbf{y} = (\mathbf{y^{(1)}}, \mathbf{y^{(2)}})$,
	$\mathbf{y^{(1)}} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2\mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2\mathbf{I})\right)$, $\sigma_1^2 = 0.1$, $\sigma_2^2 = 1$,
	$\mathbf{y^{(2)}} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I})$, $\boldsymbol{\mu} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^{D+100}$, $\mathbf{y}^{(1)} \in \mathbb{R}^D$, $\mathbf{y}^{(2)} \in \mathbb{R}^{100}$
Observation	$\mathbf{y}^o = (\mathbf{y^{o,(1)}}, \mathbf{y^{o,(2)}})$, where $\mathbf{y^{o,(1)}} = (5, \dots, 5) + \epsilon$ and $\mathbf{y^{o,(2)}} = (0, \dots, 0) + \epsilon$
Posterior	$p(\boldsymbol{\theta}\|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma_1^2\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \sigma_2^2\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The MoG simulator is the same as in the introductory example, so the proof is equivalent to Section B.

C.2 MoG Benchmark - Multiple Observations

C.2.1 Base simulator

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$
Simulator	$\mathbf{y} \sim \mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \sigma^2\mathbf{I})$, $\sigma^2 = 1$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^D$
Observations	$\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o = (5, \dots, 5) + \epsilon$ for $n \in \{1, \dots, 4\}$
Posterior	$p(\boldsymbol{\theta}\|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\sigma^2\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

Proof of the posterior:

$$
\begin{aligned}
p(\boldsymbol{\theta}|\{\mathbf{y}_n^o\}) &\propto p(\boldsymbol{\theta}) \prod_n^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) && (46)\\
&\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta}, \sigma^2\mathbf{I}) && (47)\\
&\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \sigma^2\mathbf{I}) && (48)\\
&\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2\mathbf{I}) && (49)\\
&\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\sigma^2\mathbf{I}) && (50)\\
&\propto \begin{cases} p(\boldsymbol{\theta})\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\sigma^2\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} && (51)
\end{aligned}
$$

In Figure 9, we present additional results for the base simulator with multiple observations.

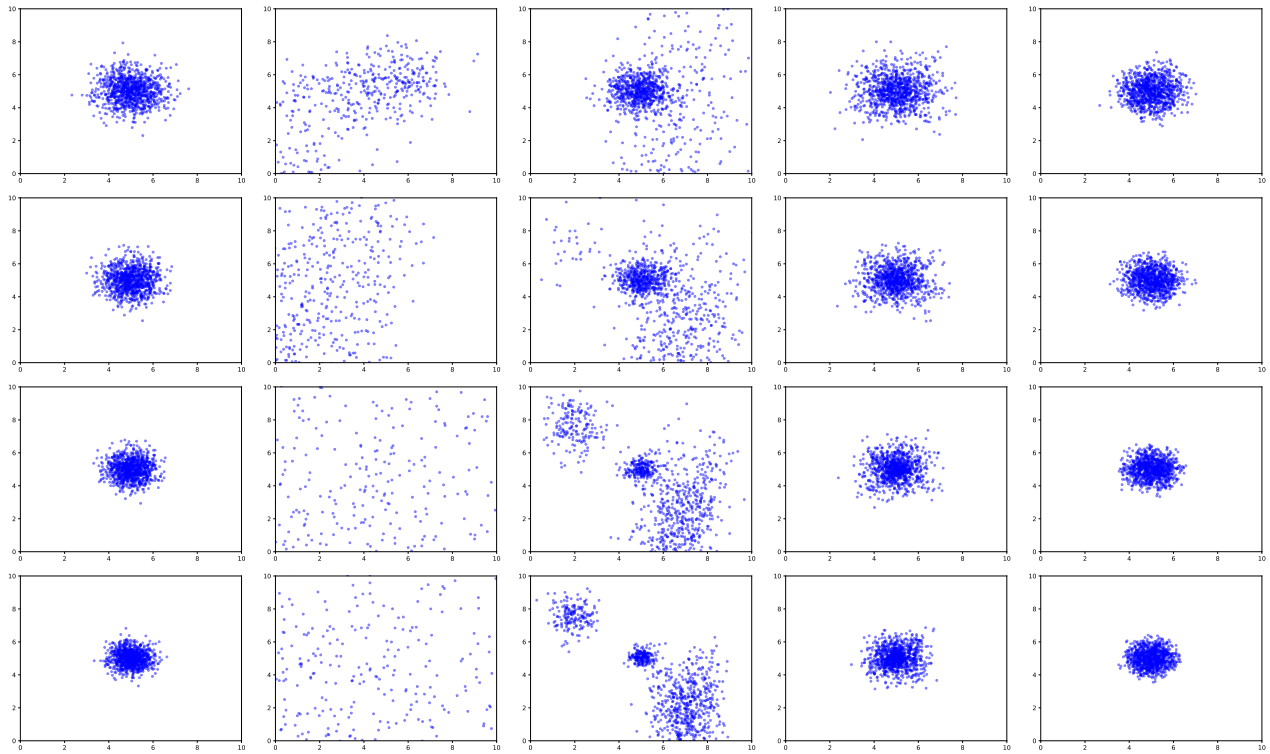


Figure 9: Multi observations Base: Left to right: ground truth, NPE, NLE, Naive-ROMC, R2OMC. Top to bottom: $N = 3$, $N = 5$, $N = 10$, $N = 10$

C.2.2 MoG simulator

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{15}, \mathbf{15})$

Simulator $\mathbf{y} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2\mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2\mathbf{I})\right)$, with $\sigma_1 = \sigma_2 = 1$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^D$

Observations $\{\mathbf{y}_n^o\}_n^N$ where:

$\mathbf{y}_n^o = (5, \dots, 5) + \epsilon$ for $n \in \{1, \dots, \frac{N}{2}\}$,

$\mathbf{y}_n^o = (-5, \dots, -5) + \epsilon$ for $n \in \{\frac{N}{2} + 1, \dots, N\}$

Posterior
$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$$

Proof of the posterior:

$$p(\boldsymbol{\theta}|\{\mathbf{y}_n^o\}) \propto\!\!\!\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} \tag{52}$$

The proof is similar to the one in Eq. (35).

In Figure 10, we present the results for the MoG simulator with multiple observations.

C.3 SBI - Mixtures of Gaussians (T.7)

We here provide additional information on example T.7 of the SBI benchmark Lueckmann et al. (2021). It tests inference of the mean of a mixture of two 2-D Gaussians, one with much broader covariance than the other

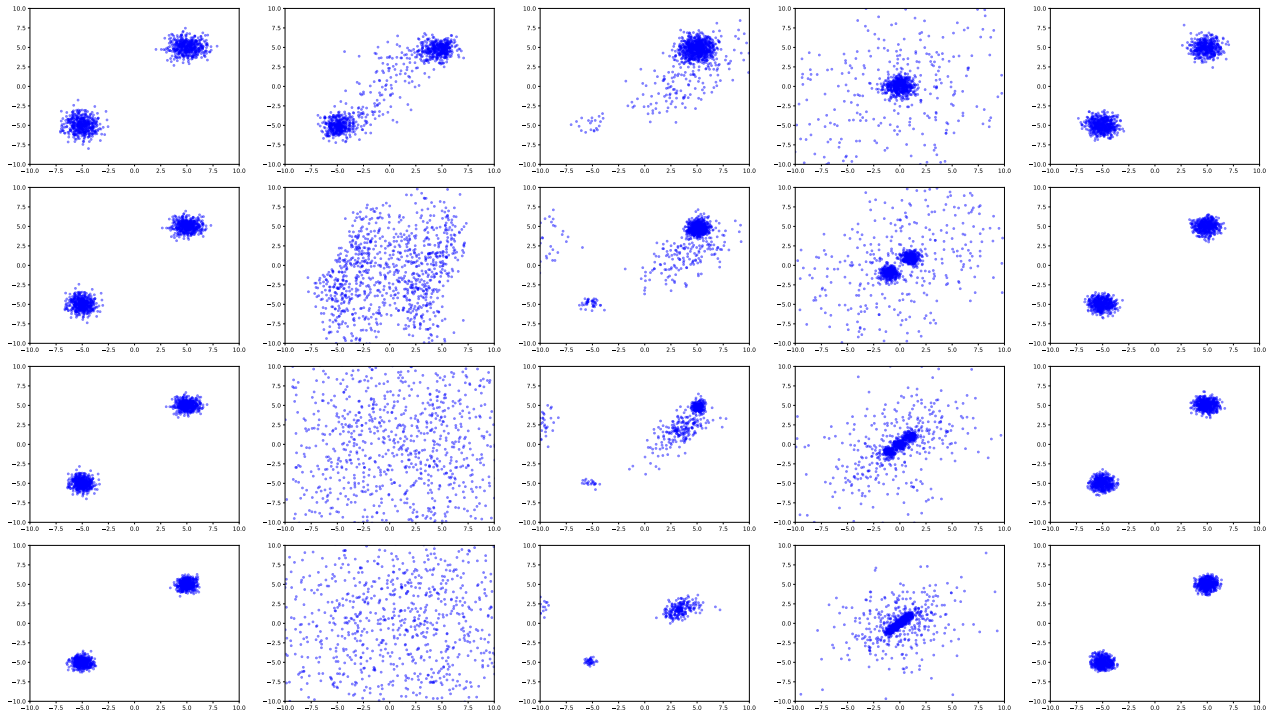
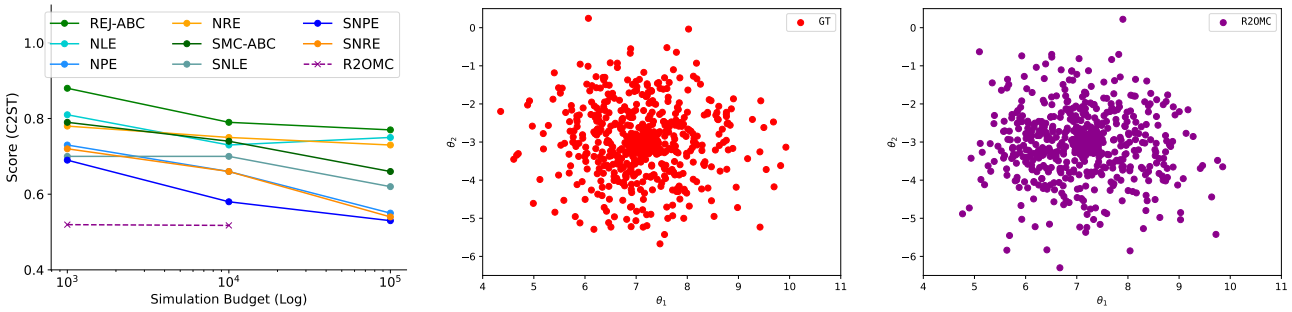


Figure 10: Multi observations MoG: Left to right: ground truth, NPE, NLE, Naive-ROMC, R2OMC. Top to bottom: $N = 3$, $N = 5$, $N = 10$, $N = 10$

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$
Simulator	$\mathbf{y} \sim 0.5(\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}, \sigma^2\mathbf{I}))$, with $\sigma^2 = 0.01$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^2$, $\mathbf{y} \in \mathbb{R}^2$

Results. We evaluate R2OMC using a simulation budget of $S = 1000$ and $S = 10000$ seeds. The `C2ST` score is computed by comparing 1000 samples from the true posterior (we randomly select 1000 from the 10000 that are provided by the SBI benchmark) with 1000 samples generated from the R2OMC approximate posterior. The results are illustrated in Figure 11.



(a) `C2ST` score (b) Ground truth samples (c) Samples by R2OMC

Figure 11: T.7: Gaussian Mixture. From left to right: `C2ST` score, ground truth samples, and samples by R2OMC.

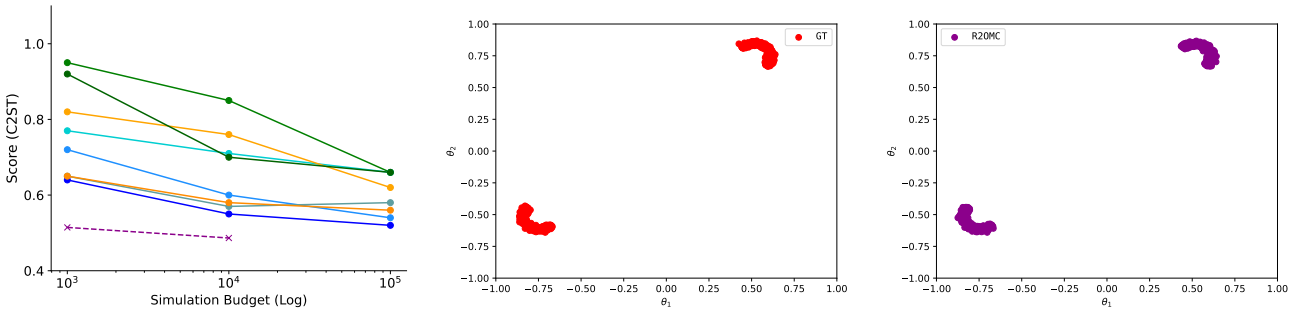
C.4 SBI - Two Moons (T.8)

We here provide additional information on example T.8 of the SBI benchmark Lueckmann et al. (2021). It tests inference when the posterior exhibits both global (bimodality) and local (crescent shape) structure to illustrate

how algorithms deal with multimodality:

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-\mathbf{1}, \mathbf{1})$
Simulator	$\mathbf{y} \sim \begin{bmatrix} r\cos(\alpha) + 0.25 \\ r\sin(\alpha) \end{bmatrix} + \begin{bmatrix} \lvert\theta_1 + \theta_2\rvert/\sqrt{2} \\ (-\theta_1 + \theta_2)/\sqrt{2} \end{bmatrix}$, where $r \sim \mathcal{N}(0.1, 0.01^2)$ and $\alpha \sim \mathcal{U}(-\pi/2, \pi/2)$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^2$, $\mathbf{y} \in \mathbb{R}^2$

Results. We evaluate R2OMC using a simulation budget of $S = 1000$ and $S = 10000$ seeds. The `C2ST` score is computed by comparing 1000 samples from the true posterior (we randomly select 1000 from the 10000 that are provided by the SBI benchmark) with 1000 samples generated from the R2OMC approximate posterior. The results are illustrated in Figure 11. We observe that R2OMC is able to capture the bimodal structure of the posterior, perfectly replicating the ground truth samples.



(a) `C2ST` score (b) Ground truth samples (c) Samples by R2OMC

Figure 12: T.8: Two Moons. From left to right: `C2ST` score, ground truth samples, and samples by R2OMC.

D Additional results

We here present some additional experiments that were not included in the main text.

D.1 MoG - Shifted Gaussian

This additional experiment uses a simulator that samples from a mixture of two Gaussians, whose means are shifted by a constant value.

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)^4$

Simulator $\mathbf{y} \sim \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta} + \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}, \mathbf{I})\right)$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^4$, $\mathbf{y} \in \mathbb{R}^4$

We will use two separate cases: one with $N = 4$ similar observations and another with $N = 4$ different observations. In both cases, the posterior is:

$$
\begin{aligned}
p(\boldsymbol{\theta}|\{\mathbf{y}_n^o\}) &\propto p(\boldsymbol{\theta}) p(\{\mathbf{y}_n^o\}|\boldsymbol{\theta}) && (53)\\
&\propto p(\boldsymbol{\theta}) \prod_{n=1}^{N} p(\mathbf{y}_n^o|\boldsymbol{\theta}) && (54)\\
&\propto \mathcal{U}(-\mathbf{15}, \mathbf{15}) \prod_{n=1}^{N} \frac{1}{2}\left(\mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta} + \mathbf{5}, \mathbf{I}) + \mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta}, \mathbf{I})\right) && (55)\\
&\propto \mathcal{U}(-\mathbf{15}, \mathbf{15}) \prod_{n=1}^{N} \frac{1}{2}\left(\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})\right) && (56)\\
&\propto\!\!\propto \prod_{n=1}^{N} \left(\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})\right) && (57)
\end{aligned}
$$

In (54), the factorization is due to the N independently and identically distributed (iid) data points. In (56), we apply the property: $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta} + \boldsymbol{\alpha}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; \mathbf{y} - \boldsymbol{\alpha}, \Sigma)$. In (57), we use that the data that we will use are sufficiently far from the boundaries so that we may ignore the boundary effects in the truncated distribution.

Case 1: In Case 1, we use $N = 4$ similar observations: $\mathbf{y}_n^o \approx (0, \ldots, 0)$ for $n \in \{1, \ldots, N\}$. The ground-truth posterior is approximately:

$$
p(\boldsymbol{\theta}|\mathbf{y}^o) \approx \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) \tag{58}
$$

Proof.

$$
\begin{aligned}
p(\boldsymbol{\theta}|\mathbf{y}_n^o) &\propto\!\!\propto \prod_{n}^{N} \left(\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})\right) && (59)\\
&\propto\!\!\propto \left(\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})\right)^N && (60)\\
&\propto\!\!\propto \sum_{n=0}^{N} \binom{N}{n} \left(\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})\right)^n \left(\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})\right)^{N-n} && (61)\\
&\propto\!\!\propto \left(\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})\right)^N + \left(\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})\right)^N && (62)\\
&\propto\!\!\propto \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) && (63)
\end{aligned}
$$

In (62), for $n \in \{1, \ldots, N-1\}$ the terms involve products between $\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})$ and $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})$ so they have a much smaller magnitude compared to $\left(\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})\right)^N$ $(n = N)$ and $\mathcal{N}((\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^N$ $(n = 0)$. This allows us to approximate the expression by retaining only these two dominant terms. In (63), we use the property that raising a Gaussian distribution to the power N results in a Gaussian with the same mean and a covariance scaled by $\frac{1}{N}$.

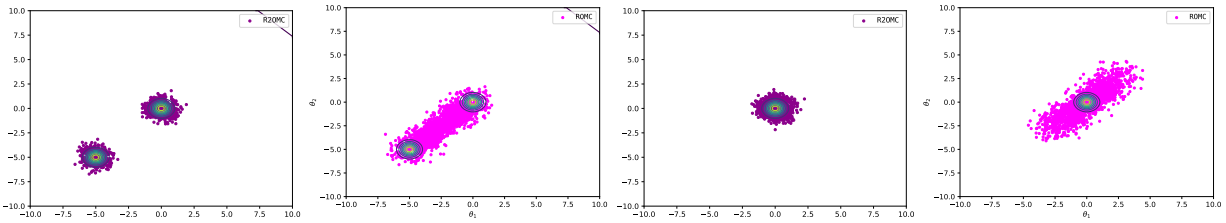


Figure 13: From left to right: R2OMC on Case 1, Naive-ROMC on Case 1, R2OMC on Case 2, and Naive-ROMC on Case 2.

Case 2: In Case 2, we use an even number of N observations that are split in two categories, half around $(0, \ldots, 0)$, i.e., $\mathbf{y}_n^o \approx (0, \ldots, 0)$ for $n \in \{1, \ldots, N/2\}$ and the other half around $(5, \ldots, 5)$, i.e., $\mathbf{y}_n^o \approx (5, \ldots, 5)$ for $n \in \{N/2+1, \ldots, N\}$. The posterior, for values of $\boldsymbol{\theta}$ sufficienty far from the boundary, is:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \approx \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) \tag{64}$$

Proof.

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto \prod_n^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})) \tag{65}$$

$$\propto \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})) \prod_{n=N/2+1}^{N} (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})) \tag{66}$$

$$\propto \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})) \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})) \tag{67}$$

$$\propto \left((\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^{N/2} + (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^{N/2}\right) \left((\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^{N/2} + (\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}))^{N/2}\right) \tag{68}$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^N \tag{69}$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) \tag{70}$$

In (68), we keep only the dominant terms, as in Case 1. In (69), we use that terms that involve combinations of $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}$, $\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})^{N/2}$ and $\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})^{N/2}$ have significantly smaller magnitudes compared to the dominant ones; $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}$. In (70), we use the property that raising a Gaussian distribution to the power N results in a Gaussian with the same mean and a covariance scaled by $\frac{1}{N}$.

Results. Figure 13 demonstrates that R2OMC aligns closely with the ground truth posterior, correctly capturing the two modes in Case 1 and the single mode in Case 2. In contrast, both ROMC struggles with multiple observations, resulting in their samples being distributed along the 45° line.